

Strings — Fundamentals & Character Access

Section 4 — JavaScript Fundamentals

Doc 4.1 — String Fundamentals & Character Access

1. What is a String?

JavaScript-এ **String** হলো text বা character-এর sequence।

যেমন:

```
const name = "Shariar";
const message = "Hello World";
const country = 'Bangladesh';
```

String-এর মধ্যে থাকতে পারে:

- Letters
- Numbers
- Spaces
- Symbols
- Special characters
- Emojis

```
const text = "Hello 123!";
const emoji = "🚀";
const address = "Rangpur, Bangladesh";
```

Important

String দেখতে অনেকগুলো character-এর collection মনে হলেও JavaScript-এ string একটি **primitive value**।

```
const name = "Shariar";

console.log(typeof name);
```

Output:

```
string
```

2. Different Ways to Create Strings

JavaScript-এ সাধারণত ৩ভাবে string লিখি।

Single Quotes

```
const name = 'Shariar';
```

Double Quotes

```
const name = "Shariar";
```

Backticks

```
const name = `Shariar`;
```

Backticks বা template literals আমরা পরের Doc-এ বিস্তারিত শিখব।

3. Single Quote vs Double Quote

দুটোর behavior প্রায় একই।

```
const firstName = "Shariar";  
const lastName = 'Mahmud';
```

দুটিই string।

```
console.log(typeof firstName); // string  
console.log(typeof lastName); // string
```

কখন কোনটা?

Project-এ সাধারণত একটি consistent style follow করা ভালো।

যেমন:

```
const name = "Shariar";  
const city = "Rangpur";
```

4. Quotes-এর ভিতরে Quote

এক ধরনের quote-এর ভিতরে অন্য quote সহজেই ব্যবহার করা যায়।

```
const message = "I'm learning JavaScript";
```

অথবা:

```
const message = 'He said "Hello"';
```

কিন্তু একই quote ব্যবহার করলে escape করতে হবে।

```
const message = "He said \"Hello\"";
```

অথবা:

```
const message = 'It\'s a good day';
```

5. Escape Characters

String-এর ভিতরে special character লেখার জন্য **escape sequence** ব্যবহার করা যায়।

Common Escape Characters

Escape	Meaning
\'	Single quote
\"	Double quote
\	Backslash
\n	New line
\t	Tab

Example:

```
const message = "Hello\nWorld";

console.log(message);
```

Output:

```
Hello
World
```

আরেকটি:

```
const text = "Name:\tShariar";

console.log(text);
```

Output:

```
Name:   Shariar
```

6. String Indexing

JavaScript string-এর প্রতিটি character-এর একটি **index** থাকে।

Index শুরু হয় **0** থেকে।

ধরো:

```
const name = "Shariar";
```

এর index হবে:

```
S h a r i a r
0 1 2 3 4 5 6
```

তাই:

```
console.log(name[0]); // s
console.log(name[1]); // h
console.log(name[2]); // a
console.log(name[3]); // r
```

7. Why Does Index Start at 0?

Programming-এ indexing সাধারণত zero-based হয়।

```
const word = "Hello";
```

Position:

```
H → index 0
e → index 1
l → index 2
l → index 3
o → index 4
```

Total character:

```
word.length
```

returns:

```
5
```

কিন্তু last index:

```
4
```

Formula

```
Last index = length - 1
```

Example:

```
const word = "Hello";

console.log(word.length); // 5
console.log(word[4]);      // o
```

8. Accessing Characters Using `[]`

সবচেয়ে সহজ character access:

```
const name = "Shariar";

console.log(name[0]);
console.log(name[3]);
console.log(name[6]);
```

Output:

```
S
r
r
```

9. Invalid Index

যদি এমন index access করি যেটা string-এর মধ্যে নেই:

```
const name = "Shariar";

console.log(name[100]);
```

Output:

```
undefined
```

Negative index:

```
console.log(name[-1]);
```

এটাও:

```
undefined
```

কারণ traditional bracket indexing negative index support করে না।

Negative indexing-এর জন্য আমরা `at()` ব্যবহার করতে পারি।

10. `length` Property

String-এর মোট কতগুলো code unit আছে সেটা জানতে `.length` ব্যবহার করি।

```
const name = "Shariar";

console.log(name.length);
```

Output:

```
7
```

আরেকটি:

```
const message = "Hello World";

console.log(message.length);
```

Output:

```
11
```

এখানে space-ও count হয়েছে।

```
Hello World
      ↑
    space
```

তাই:

```
Hello = 5
  space = 1
World = 5

Total = 11
```

11. `length` হলো Property, Method নয়

এটা খুব important!

```
name.length
```

এখানে `length` একটি **property**।

তাই parentheses লাগে না।

❌ ভুল:

```
name.length();
```

✅ সঠিক:

```
name.length;
```

অন্যদিকে:

```
name.toUpperCase();
```

এটা method, তাই `()` লাগে।

12. String-এর Last Character

String-এর last character পেতে:

```
const name = "Shariar";

console.log(name[name.length - 1]);
```

Output:

```
r
```

কেন?

```
name.length
```

হলো:

```
7
```

Last index:

```
7 - 1 = 6
```

তাই:

```
name[6]
```

হলো:

```
r
```

13. `String.prototype.at()`

Modern JavaScript-এ character access করার জন্য `.at()` method ব্যবহার করা যায়।

Syntax:

```
string.at(index)
```

Example:

```
const name = "Shariar";

console.log(name.at(0)); // S
console.log(name.at(3)); // r
console.log(name.at(6)); // r
```

14. `at()` vs `[]`

দুটো দিয়েই normal positive index access করা যায়।

```
const name = "Shariar";

console.log(name[2]);
console.log(name.at(2));
```

দুটোই:

```
a
```

কিন্তু `at()` -এর একটি important advantage আছে।

Negative Index Support

```
const name = "Shariar";

console.log(name.at(-1));
```

Output:

```
r
```

`-1` মানে last character।

```
Shariar
      ↑
     -1
```

আর:

```
console.log(name.at(-2));
```

Output:

```
a
```

15. Negative Indexing with `at()`

এটা মনে রাখো:

```
at(-1) → last character  
at(-2) → second last  
at(-3) → third last
```

Example:

```
const word = "JavaScript";  
  
console.log(word.at(-1)); // t  
console.log(word.at(-2)); // p  
console.log(word.at(-3)); // i
```

এটা last character access অনেক cleaner করে।

Traditional approach:

```
word[word.length - 1];
```

Modern approach:

```
word.at(-1);
```

16. `at()` with Out-of-Range Index

যদি index exist না করে:

```
const word = "Hello";

console.log(word.at(100));
```

Output:

```
undefined
```

Negative হলেও range-এর বাইরে গেলে:

```
console.log(word.at(-100));
```

Output:

```
undefined
```

17. `at()` Works with Arrays Too

`.at()` শুধু string-এর জন্য নয়।
Array-এর সাথেও কাজ করে।

```
const fruits = ["apple", "banana", "mango"];

console.log(fruits.at(0)); // apple
console.log(fruits.at(-1)); // mango
```

তাই `.at()` -কে general **index-based access method** হিসেবে চিনতে পারো।

18. String Immutability

এটা JavaScript String-এর সবচেয়ে important concept-এর একটি।

Strings are immutable.

এর অর্থ:

একবার একটি string value তৈরি হলে, সেই existing string-এর individual character সরাসরি পরিবর্তন করা যায় না।

19. String Character Directly Change করা যায় না

ধরো:

```
let name = "Shariar";

name[0] = "X";

console.log(name);
```

তুমি ভাবতে পারো:

```
Xhariar
```

কিন্তু হবে:

```
Shariar
```

কারণ string immutable।

20. তাহলে String Change করা হয় কীভাবে?

Existing string modify না করে নতুন **string** তৈরি করতে হয়।

Example:

```
let name = "Shariar";

name = "X" + name.slice(1);

console.log(name);
```

Output:

```
Xhariar
```

এখানে পুরোনো "Shariar" modify হয়নি।

বরং নতুন string তৈরি হয়েছে:

```
"Xhariaar"
```

এবং `name` variable-কে নতুন string-এর দিকে point করানো হয়েছে।

21. Variable Reassignment vs String Mutation

এই দুইটা আলাদা বিষয়।

Reassignment

```
let name = "Shariar";  
  
name = "Duke";
```

এটা valid।

কারণ আমরা variable-এর value reference পরিবর্তন করছি।

Mutation

```
name[0] = "X";
```

এটা string-এর individual character পরিবর্তন করার চেষ্টা।

String immutable হওয়ায় এটা কাজ করবে না।

22. `const` থাকলে কী হয়?

এখানে আরেকটা interesting বিষয় আছে।

```
const name = "Shariar";  
  
name = "Duke";
```

এটা error দেবে।

কারণ `const` variable reassignment allow করে না।

কিন্তু এর সাথে string immutability এক জিনিস নয়।

মনে রাখবে

```
const → variable reassignment control করে
```

```
String immutability → string value mutation control করে
```

দুটো completely different concepts।

23. Mental Model of String Immutability

ধরো:

```
let name = "Shariar";
```

মনে করো memory-তে:

```
name  
↓  
"Shariar"
```

এখন:

```
name = "Duke";
```

হলো:

```
name  
↓  
"Duke"
```

পুরোনো `"Shariar"` string-কে modify করা হয়নি।

Variable নতুন string value-এর দিকে point করছে।

24. Common String Operations Create New Strings

String-এর অনেক operation existing string modify করে না।

বরং নতুন string return করে।

যেমন:

```
const name = "shariar";

const upper = name.toUpperCase();

console.log(name);
console.log(upper);
```

Output:

```
shariar
SHARIAR
```

Original:

```
shariar
```

unchanged আছে।

25. Why String Immutability Matters

String immutable হওয়ায় তুমি safely একই string value বিভিন্ন জায়গায় ব্যবহার করতে পারো।

```
const username = "Shariar";

const message1 = `Hello ${username}`;
const message2 = `Welcome ${username}`;

console.log(message1);
console.log(message2);
```

এক জায়গায় operation করার কারণে অন্য জায়গার original string পরিবর্তন হয় না।

26. String Primitive vs **String** Object

JavaScript-এ দুই ধরনের string-related value দেখতে পারো।

Primitive String

```
const name = "Shariar";
```

এটাই normal এবং recommended।

String Object

```
const name = new String("Shariar");
```

এটা একটি object।

```
console.log(typeof name);
```

Output:

```
object
```

অন্যদিকে:

```
const name = "Shariar";  
  
console.log(typeof name);
```

Output:

```
string
```

27. Why Should You Usually Avoid `new String()` ?

সাধারণ string-এর জন্ম:

```
const name = "Shariar";
```

ব্যবহার করাই ভালো।

অপ্রয়োজনে:

```
const name = new String("Shariar");
```

ব্যবহার করলে primitive string-এর পরিবর্তে object তৈরি হয়, যা equality এবং behavior-এর ক্ষেত্রে confusion তৈরি করতে পারে।

Example:

```
const a = "hello";  
const b = new String("hello");  
  
console.log(a === b);
```

Output:

```
false
```

কারণ:

```
a → primitive string  
  
b → String object
```

Best Practice

```
const name = "Shariar";
```

Use `new String()` সাধারণ application code-এ প্রয়োজন হয় না।

28. Strings and Numbers

String-এর মধ্যে number-এর মতো দেখতে data থাকতে পারে।

```
const age = "31";
```

এখানে `31` number নয়।

```
console.log(typeof age);
```

Output:

```
string
```

অন্যদিকে:

```
const age = 31;

console.log(typeof age);
```

Output:

```
number
```

এই distinction পরে type coercion এবং form/API data handling-এর সময় খুব গুরুত্বপূর্ণ হবে।

29. Empty String

Empty string:

```
const name = "";
```

এর length:

```
console.log(name.length);
```

Output:

```
0
```

কিন্তু:

```
const name = " ";
```

এটা empty নয়।

এখানে একটি space আছে।

```
console.log(name.length);
```

Output:

```
1
```

এটা পরবর্তীতে form validation-এর সময় খুব important হবে।

30. Strings Can Contain Spaces

```
const fullName = "Shariar Mahmud Duke";
```

এখানে spaces-ও string-এর অংশ।

```
console.log(fullName.length);
```

প্রতিটি space count হবে।

31. Unicode and Emojis — Important Note

JavaScript string-এর `.length` সবসময় visible character-এর সংখ্যা বোঝায় না।

কারণ JavaScript internally string-কে **UTF-16 code units** হিসেবে handle করে।

সাধারণ English character:

```
console.log("A".length);
```

Output:

```
1
```

কিন্তু কিছু emoji:

```
console.log("👨".length);
```

Output:

```
2
```

যদিও visually আমরা এটাকে একটি emoji হিসেবে দেখি।

তাই:

```
string.length
```

কে সবসময় "মানুষের চোখে দেখা character count" হিসেবে ভাবা ঠিক নয়।

এই বিষয়টা Unicode, UTF-16 এবং grapheme clusters শেখার সময় আরও গভীরভাবে দেখা যায়।

32. `at()` and Unicode

এখানেও একটা important distinction আছে।

```
const emoji = "👨‍👩‍👧";  
  
console.log(emoji.at(0));
```

এটা পুরো emoji return করার মতো behavior নাও দিতে পারে, কারণ `at()` UTF-16 code unit-এর ভিত্তিতে কাজ করে।

যখন তুমি সত্যিকারের user-perceived character বা grapheme নিয়ে কাজ করবে, তখন `Array.from()` বা `Intl.Segmenter`-এর মতো tools relevant হতে পারে।

Example:

```
const emoji = "👨‍👩‍👧";  
  
console.log(Array.from(emoji).length);
```

Output:

```
1
```

Beginner level-এ মূল বিষয়:

সাধারণ English text-এর জন্য `length`, indexing এবং `at()` খুব straightforward; Unicode/emoji edge cases আলাদা বিষয়।

33. Common Mistakes

Mistake 1 — Index starts from 1 ভাবা



```
const name = "Shariar";

console.log(name[1]); // ভাবে S
```

Actually:

```
h
```

কারণ:

```
S → 0
h → 1
```

Mistake 2 — Last index = length



```
const name = "Hello";

console.log(name[name.length]);
```

এটা:

```
undefined
```

কারণ:

```
length = 5
last index = 4
```

সঠিক:

```
console.log(name[name.length - 1]);
```

Mistake 3 — String character modify করা



```
let name = "Hello";  
  
name[0] = "Y";
```

String immutable!

Mistake 4 — `length()` লেখা



```
name.length();
```



```
name.length;
```

Mistake 5 — `new String()` অপ্রয়োজনে ব্যবহার করা



```
const name = new String("Shariar");
```

সাধারণ ক্ষেত্রে:

```
const name = "Shariar";
```

34. Practical Example — Get Username Initial

```
const username = "Shariar";

const initial = username.at(0);

console.log(initial);
```

Output:

```
S
```

35. Practical Example — Get Last Character

```
const username = "Shariar";

const lastCharacter = username.at(-1);

console.log(lastCharacter);
```

Output:

```
r
```

36. Practical Example — Check Username Length

```
const username = "shariar123";

if (username.length >= 5) {
  console.log("Valid username");
} else {
  console.log("Username is too short");
}
```

37. Practical Example — Get File Extension Character

```
const fileName = "profile.jpg";

console.log(fileName.at(-1));
```

Output:

g

বাস্তবে file extension বের করার জন্য আমরা পরে `split()`, `slice()` ইত্যাদি ব্যবহার করতে পারি।

38. `[]` vs `at()` — Quick Comparison

Feature	<code>str[index]</code>	<code>str.at(index)</code>
Positive index	✓	✓
Negative index	✗	✓
Out of range	<code>undefined</code>	<code>undefined</code>
Syntax	Simple	Method call
Last character	<code>str[str.length - 1]</code>	<code>str.at(-1)</code>

Example

```
const word = "JavaScript";

console.log(word[0]);
console.log(word.at(0));

console.log(word.at(-1));
```

Output:

```
J
J
t
```

39. Key Mental Model

String-কে এভাবে ভাবো:

```
"JavaScript"
```

```
J a v a S c r i p t
0 1 2 3 4 5 6 7 8 9
```

Access

```
word[0]
word.at(0)
```

Last character

```
word[word.length - 1]
word.at(-1)
```

Length

```
word.length
```

Modification?

```
word[0] = "X";
```

✗ Direct mutation করা যায় না।

New string তৈরি?

```
const newWord = "X" + word.slice(1);
```

40. Practice — Basic

Question 1

Output কী হবে?

```
const word = "JavaScript";

console.log(word[0]);
console.log(word[4]);
console.log(word.length);
```

Question 2

Output কী হবে?

```
const name = "Shariar";

console.log(name.at(-1));
console.log(name.at(-2));
console.log(name.at(-3));
```

Question 3

Output কী হবে?

```
const text = "Hello";

console.log(text[text.length - 1]);
console.log(text[text.length]);
```

Question 4

Output কী হবে?

```
const text = "";

console.log(text.length);
```

Question 5

Output কী হবে?

```
const text = " ";

console.log(text.length);
```

41. Practice — Immutability

এই code-এর output কী হবে?

```
let word = "Hello";

word[0] = "Y";

console.log(word);
```

তারপর explain করো কেন এমন output হলো।

42. Coding Challenge 1 — First Character

একটি function তৈরি করো যেটা একটি string নেবে এবং তার first character return করবে।

Example:

```
getFirstCharacter("JavaScript");
```

Expected:

J

43. Coding Challenge 2 — Last Character

একটি function তৈরি করো যেটা string-এর last character return করবে।

Example:

```
getLastCharacter("JavaScript");
```

Expected:

```
t
```

Try করার সময় `at()` ব্যবহার করো।

44. Coding Challenge 3 — String Length

Function:

```
getStringLength("Bangladesh");
```

Expected:

```
10
```

45. Coding Challenge 4 — First and Last Character

Function:

```
getFirstAndLast("JavaScript");
```

Expected:

```
{  
  first: "J",  
  last: "t"  
}
```

46. Interview Questions

Beginner

1. What is a string in JavaScript?
2. How many ways can you create a string?
3. What is string indexing?
4. Why does JavaScript indexing start from `0`?
5. What does `.length` return?
6. Is `.length` a method or property?
7. What happens when you access an invalid string index?
8. What is `String.prototype.at()`?
9. What advantage does `at()` have over bracket notation?
10. How do you get the last character of a string?

Intermediate

11. Are JavaScript strings mutable or immutable?
 12. What happens when you try to change `str[0]`?
 13. What is the difference between reassignment and mutation?
 14. What is the difference between a primitive string and `new String()`?
 15. Why is `new String()` generally avoided?
 16. Why can `"😄".length` be greater than `1`?
-

47. Quick Revision

```
String
  ↓
Text / sequence of UTF-16 code units
  ↓
Primitive value
  ↓
Index starts from 0
```

Creation

```
"Hello"
'Hello'
`Hello`
```

Length

```
str.length
```

Character access

```
str[0]
str.at(0)
```

Last character

```
str[str.length - 1]
str.at(-1)
```

Invalid index

```
str[100]
str.at(100)
```

Result:

undefined

Immutability

```
str[0] = "X";
```

✗ Existing string পরিবর্তন হয় না।

Reassignment

```
let str = "Hello";  
  
str = "World";
```

✓ Valid।

48. Golden Rules

Rule 1: String একটি primitive data type।

Rule 2: String indexing `0` থেকে শুরু হয়।

Rule 3: `length` property, method নয়।

Rule 4: Last index = `length - 1`।

Rule 5: `at(-1)` দিয়ে সহজে last character পাওয়া যায়।

Rule 6: JavaScript strings immutable।

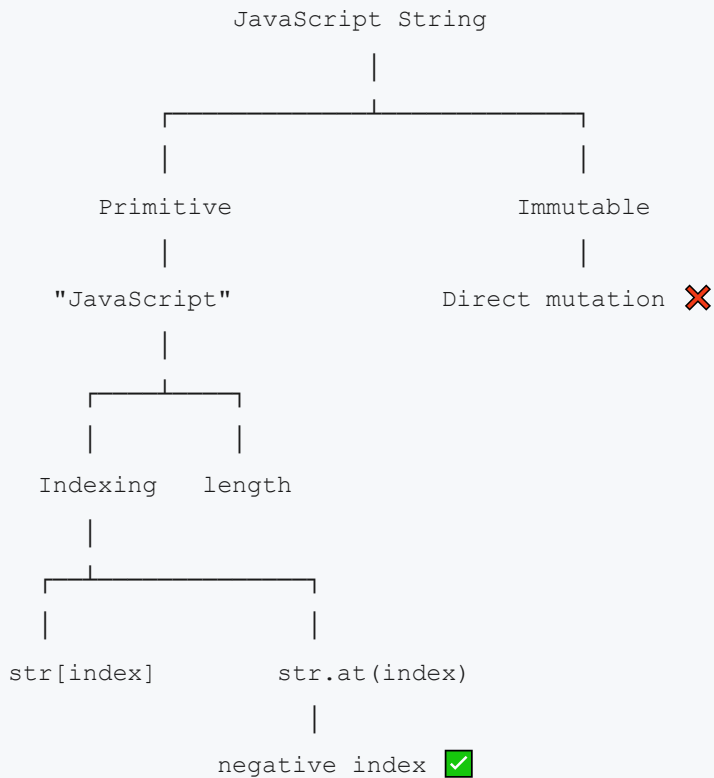
Rule 7: String-এর character সরাসরি modify করা যায় না।

Rule 8: String পরিবর্তনের মতো কিছু করতে হলে সাধারণত নতুন string তৈরি করতে হয়।

Rule 9: সাধারণ application code-এ primitive string ব্যবহার করো; `new String()` এড়িয়ে চলো।

Rule 10: `.length` সবসময় *visible human characters*-এর সংখ্যা বোঝায় না—Unicode/emoji-এর ক্ষেত্রে *UTF-16 behavior* মনে রাখতে হবে।

Final Mental Picture



এই foundation clear হলে পরের Doc-এর `indexOf()`, `includes()`, `slice()`, `split()`, `join()`, `trim()`, `replace()`, `replaceAll()`, **Template Literals**, **Tagged Templates** এবং `String.raw` অনেক সহজে বুঝতে পারবে।